

Materiale aggiuntivo su ss

Riferimento alla man page:

<https://manpages.ubuntu.com/manpages/jammy/en/man8/ss.8.html>

Stato generale delle socket nel sistema

```
$ ss -s
```

Total: 288

TCP: 9 (estab 4, closed 0, orphaned 0, timewait 0)

Transport	Total	IP	IPv6
RAW	0	0	0
UDP	14	9	5
TCP	9	9	0
INET	23	18	5
FRAG	0	0	0

Numero complessivo di socket gestite dal kernel

Numero di socket TCP, ulteriormente suddivise tra: quelle con connessione completamente stabilita; quelle chiuse ma ancora in tabella; quelle non associate a nessun processo e quelle in TIME_WAIT.

Le socket RAW permettono di bypassare i protocolli di trasporto e di costruire manualmente il payload e, opzionalmente, l'intestazione IP. Tuttavia, per operare direttamente sul livello di collegamento occorrono socket di tipo packet (es. AF_PACKET su Linux).

Frammenti di datagrammi IP (v4 e v6)

Il comando ss senza parametri elenca tutte le socket non-listening con connessione stabilita. Il comando produce una tabella le cui colonne hanno il seguente significato:

- **Netid** – Type of socket. Common types are *TCP*, *UDP*, *u_str* (Unix stream), and *u_seq* (Unix sequence).
- **State** – State of the socket. Most commonly *ESTAB* (established), *UNCONN* (unconnected), *LISTEN* (listening).
- **Recv-Q** – Number of received packets in the queue.
- **Send-Q** – Number of sent packets in the queue.
- **Local address:port** – Address of local machine and port.
- **Peer address:port** – Address of remote machine and port.

Le opzioni possono essere scritte separatamente oppure combinate:

```
ss -t -u
```

è la stessa cosa che

```
ss -tu
```

Si noti che le due opzioni vanno interpretate in “or”, nel senso che saranno mostrato socket relative al protocollo TCP oppure UDP.

```
ss -l
```

Mostra soltanto le socket listening

```
ss -a
```

mostra sia le socket listening sia quelle non-listening.

```
ss -n
```

evita di risolvere i nomi dei servizi, cioè di sostituire ai numeri di porta il nome del servizio corrispondente (/etc/services).

```
ss -p
```

mostra il processo che usa la socket: potrebbe essere necessario lanciare il comando con sudo. Restituisce una colonna aggiuntiva con valori simili al seguente:

```
users: ( ("node",pid=727,fd=22) )
```

dove riconosciamo il nome dell'eseguibile, il pid del processo e il file descriptor che rappresenta la socket.

Altra opzione interessante:

```
-i, --info
```

Show internal TCP information. Below fields may appear:

```
ts      show string "ts" if the timestamp option is set
```

```
sack    show string "sack" if the sack option is set
```

```
ecn     show string "ecn" if the explicit congestion notification  
option is set
```

```
ecnseen
```

```
show string "ecnseen" if the saw ecn flag is found in received  
packets
```

```
fastopen
```

```
show string "fastopen" if the fastopen option is set
```

cong_alg
the congestion algorithm name, the default congestion algorithm is "cubic"

wscale:<snd_wscale>:<rcv_wscale>
if window scale option is used, this field shows the send scale factor and receive scale factor

rto:<icsk_rto>
tcp re-transmission timeout value, the unit is millisecond

backoff:<icsk_backoff>
used for exponential backoff re-transmission, the actual re-transmission timeout value is $\text{icsk_rto} \ll \text{icsk_backoff}$

rtt:<rtt>/<rttvar>
rtt is the average round trip time, rttvar is the mean deviation of rtt, their units are millisecond

ato:<ato>
ack timeout, unit is millisecond, used for delay ack mode

mss:<mss>
max segment size

cwnd:<cwnd>
congestion window size

pmtu:<pmtu>
path MTU value

ssthresh:<ssthresh>
tcp congestion window slow start threshold

bytes_acked:<bytes_acked>
bytes acked

bytes_received:<bytes_received>
bytes received

segs_out:<segs_out>
segments sent out

segs_in:<segs_in>
segments received

send <send_bps>bps
egress bps

lastsnd:<lastsnd>
how long time since the last packet sent, the unit is millisecond

lastrcv:<lastrcv>

```

                                how long time since the last packet received, the unit is
millisecond

                                how long time since the last ack received, the unit is
millisecond

                                the pacing rate and max pacing rate
                                <max_pacing_rate>bps
                                <pacing_rate>bps/

                                a helper variable for TCP internal auto tuning socket receive
buffer
                                <rcv_space>
                                rcv_space:

                                flags: [MmBbJjecv]
                                tcp-ulp-mptcp
                                token:<rem_token(rem_id)/loc_token(loc_id)>
                                seq:<sn> sfseq:<ssn> ssnooff:<off> maplen:<maplen>
                                MPTCP subflow information

```

La sintassi del comando ss è:

```
ss [options] [ FILTER ]
```

Abbiamo già visto alcune opzioni, cosa possiamo dire del filter?

Si compone di due parti opzionali:

```
[ state STATE-FILTER ] [ EXPRESSION ]
```

STATE-FILTER permette di filtrare le socket usando una sequenza di identificatori di stato e esclusioni.

Possiamo usare tutti gli stati TCP:

```
established, syn-sent, syn-recv, fin-wait-1, fin-wait-2, time-wait, closed,
close-wait, last-ack, listening, closing
```

Cui si aggiungono questi identificatori:

all – per tutti gli stati

connected – tutti gli stati tranne **listening** e **closed**

synchronized – tutti gli stati **connected** tranne che **syn-sent**

bucket – stati che sono mantenuti come minisocket, es. **time-wait** e **syn-recv**

big – opposto di **bucket**

EXPRESSION permette di combinare diversi predicati usando operatori logici `and` (`o && o &`), `or` (`o || o ||`), `not` (`o !`). Se non viene indicato un operatore si assume `and`.

Alcuni predicati:

`{dst|src} [=] HOST`

Testa se la sorgente o la destinazione corrispondono all'HOST (vedi sotto).

`{dport|sport} [OP] [FAMILY:] PORT`

Confronta la porta di destinazione o sorgente con porta. OP può essere uno tra "<", "<=", "=", "!=" , ">=" and ">".

`autobound`

Testa se la porta (o il path nel caso di altri tipi di socket diverse da quelle Internet) è stata allocata dinamicamente.

HOST può essere definito come un hostname DNS o indirizzo IP secondo le convenzioni di IPv4 (dot-decimal notation) o IPv6

PORT può essere dato come numero oppure usando uno dei nomi che sono listati in `/etc/services` (che sono gli stessi nomi usati senza l'opzione `-n`).

Sia PORT sia HOST possono essere sostituiti da un asterisco (come *wildcard*).

Notate che `expression` è usato per filtrare le socket come mostrate sulla base delle altre opzioni e dello `state-filter`.

```
$ ss sport = :domain
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
-------	-------	--------	--------	--------------------	-------------------	---------

Mentre

```
$ ss -a sport = :domain
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.54:domain	0.0.0.0:*	
udp	UNCONN	0	0	127.0.0.53%lo:domain	0.0.0.0:*	
udp	UNCONN	0	0	10.255.255.254:domain	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.54:domain	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:domain	0.0.0.0:*	
tcp	LISTEN	0	1000	10.255.255.254:domain	0.0.0.0:*	

La differenza è motivata dal fatto che le socket mostrate nella seconda stampa sono socket TCP passive e socket UDP non connesse, entrambe escluse dall'elenco predefinito.

Esempi

(notate lo spazio tra le parentesi):

```
$ ss -t state established '( dport == :9000 and dst == 127/8 and autobound ) '
```

(la and è implicita se non ci sono connettivi):

```
$ ss -t state established 'dport == :9000 dst == 127/8 autobound'
```

(notate che il prefisso è stato combinato con il numero di porta)

```
$ ss -t state established '(dst == 127/8:9000 and autobound ) '
```

(notate l'uso degli asterischi come wildcard)

```
$ ss -t 'dst = *:*'
```

Gli apici servono in alcuni casi (per esempio, quando si usano le parentesi tonde e gli asterischi per evitare che siano interpretati da Bash).